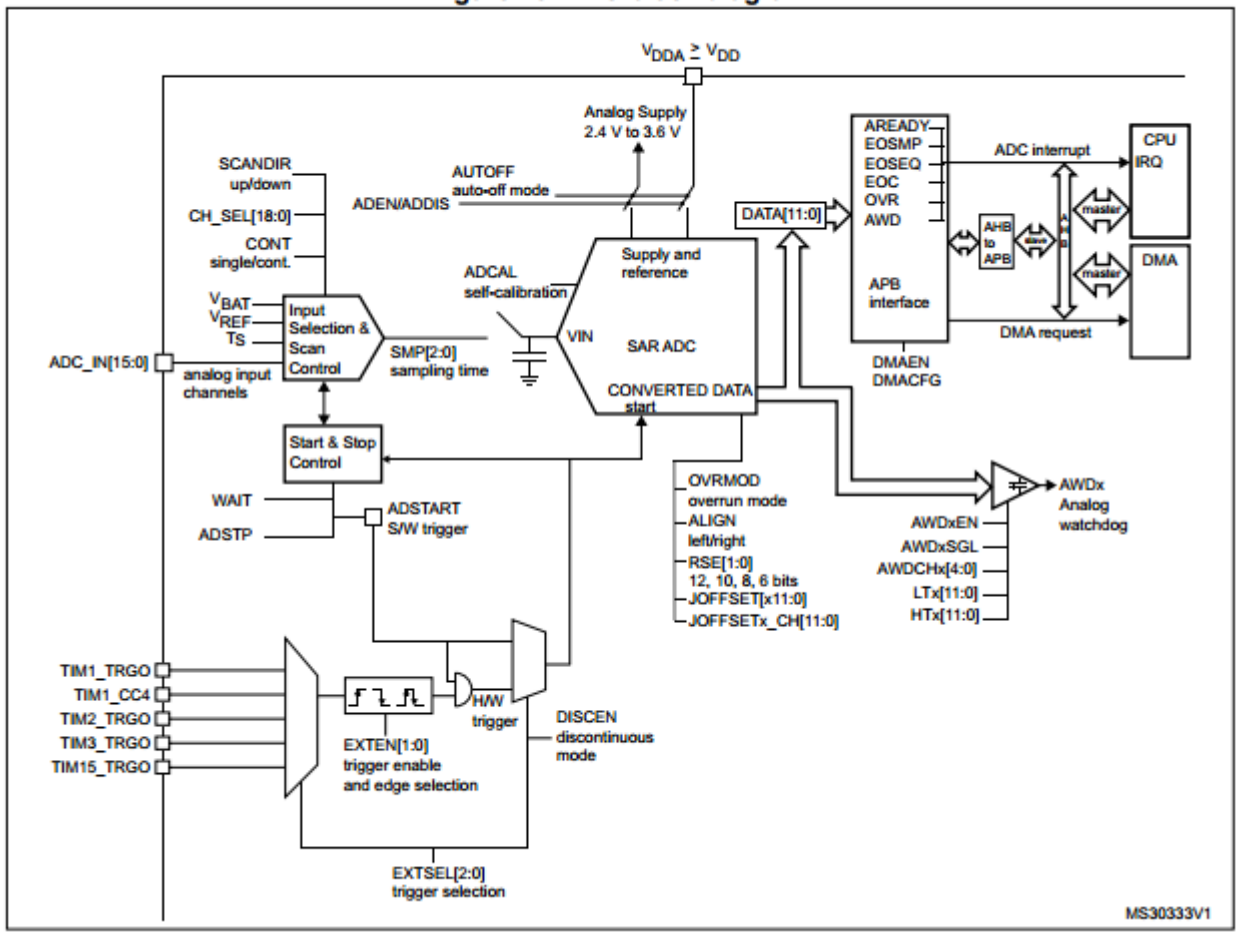
ADC

ANALOGUE TO DIGITAL CONVERTER

ADC

Figure 25. ADC block diagram



V_DD = Main CPU voltage

V_DDA = ADC reference voltage

ADC

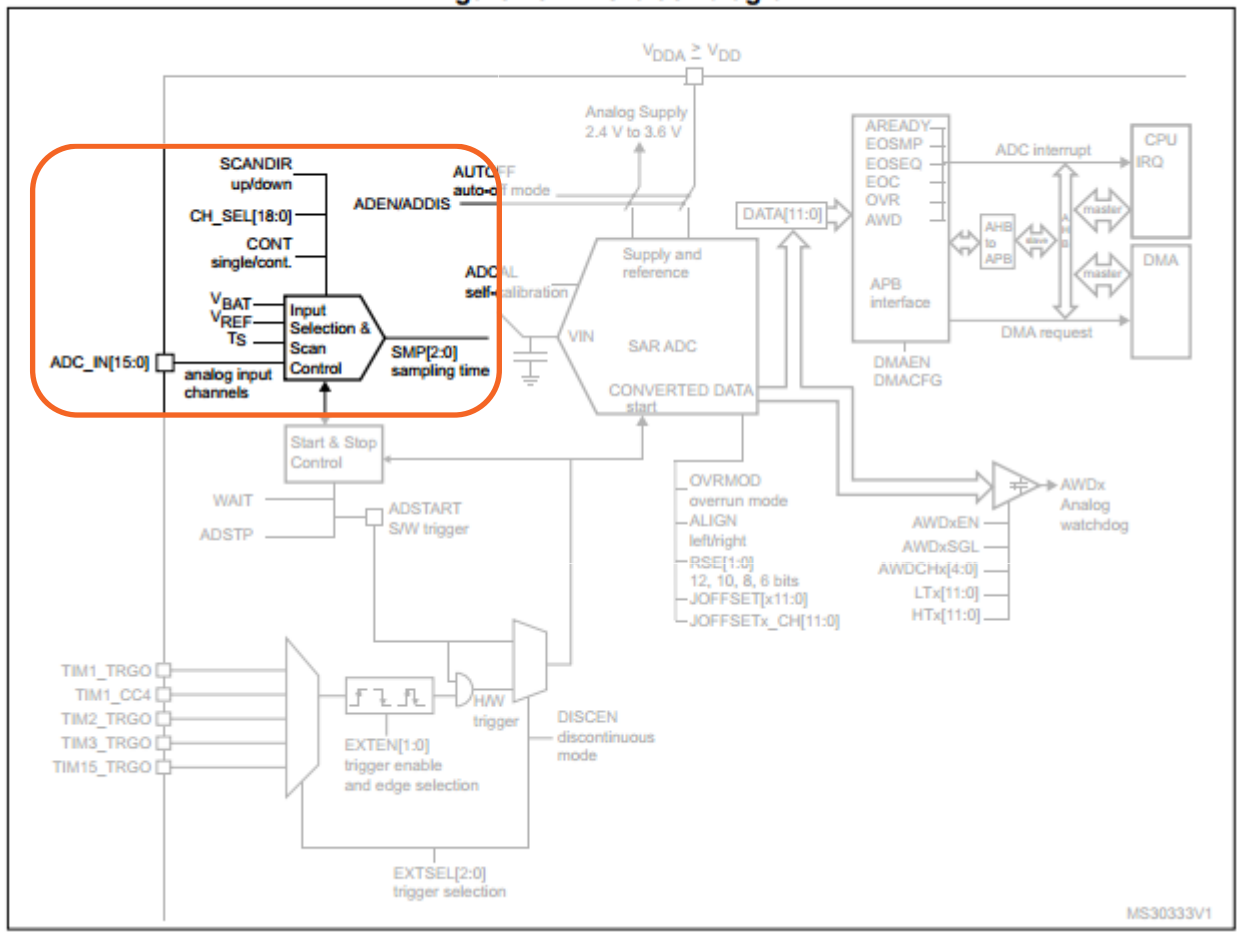
Sample Settings

- 16 channel multiplexed analogue input from GPIO – ADC_IN[15:0]
- Three internal inputs V_{BAT}, V_{REF} and T_S

Done in:

- Single or continuous conversion
- Up scan (ADC_IN0 first) or down scan (V_{BAT} first).

Figure 25. ADC block diagram

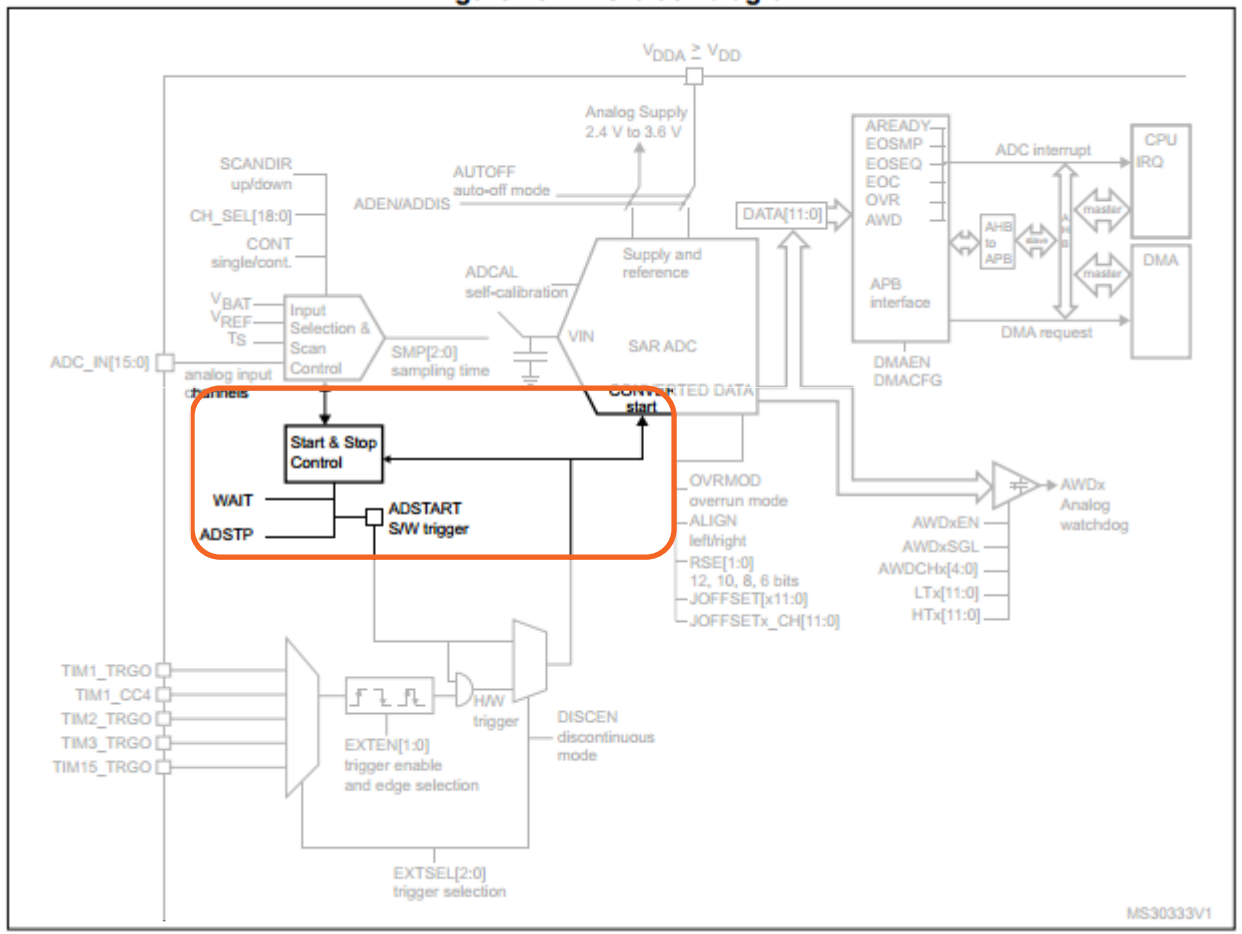


ADC

Conversion Start/Stop

- Controls starting and stopping of a sequence of conversions

Figure 25. ADC block diagram

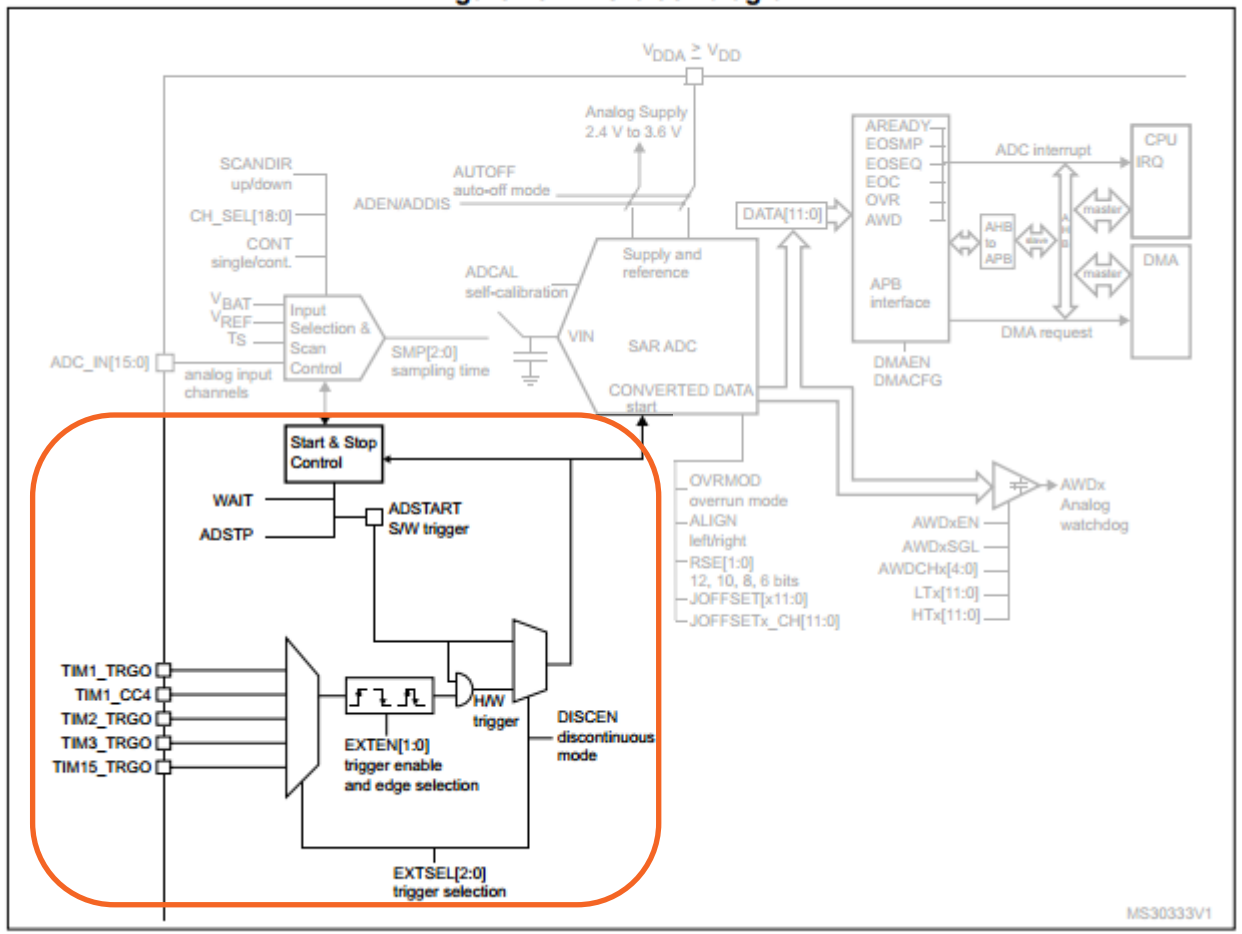


ADC

Conversion Start/Stop

- Controls starting and stopping of a sequence of conversions
- Various timers and external triggers can be used to initiate conversions as well

Figure 25. ADC block diagram

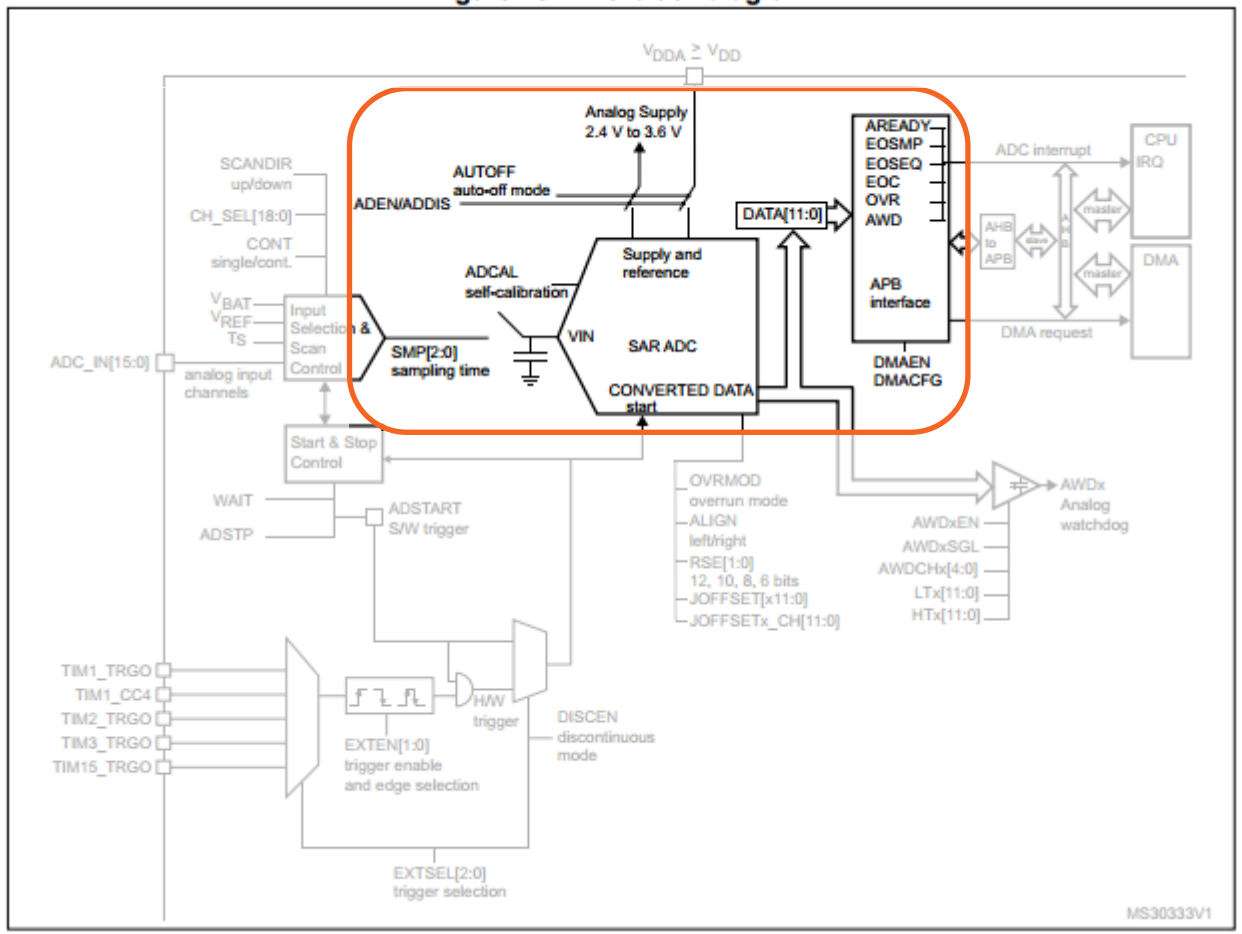


ADC

Conversion Hardware

- Implements the successive approximation hardware

Figure 25. ADC block diagram

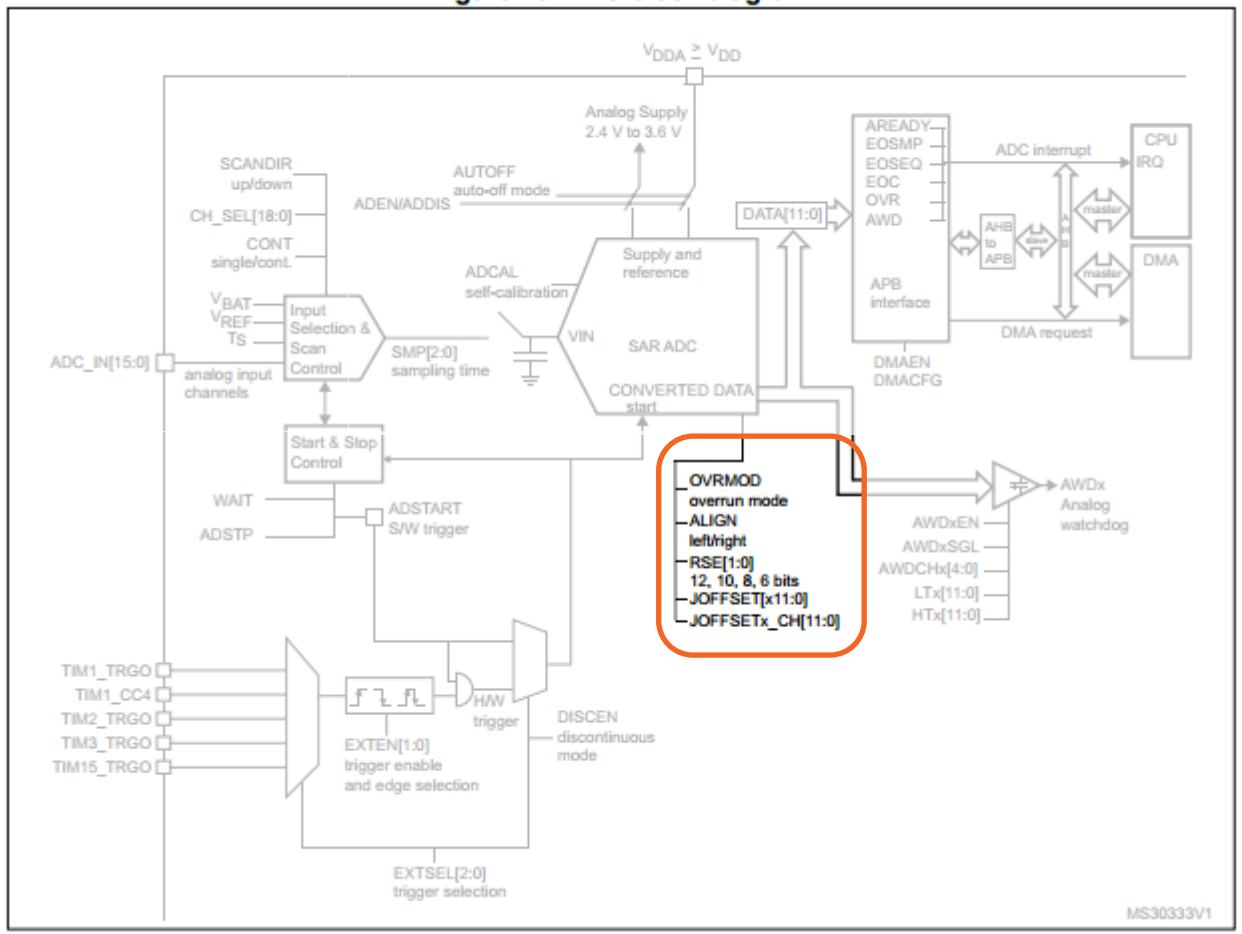


ADC

Conversion Settings

- E.g. Conversion resolution, left/right alignment

Figure 25. ADC block diagram

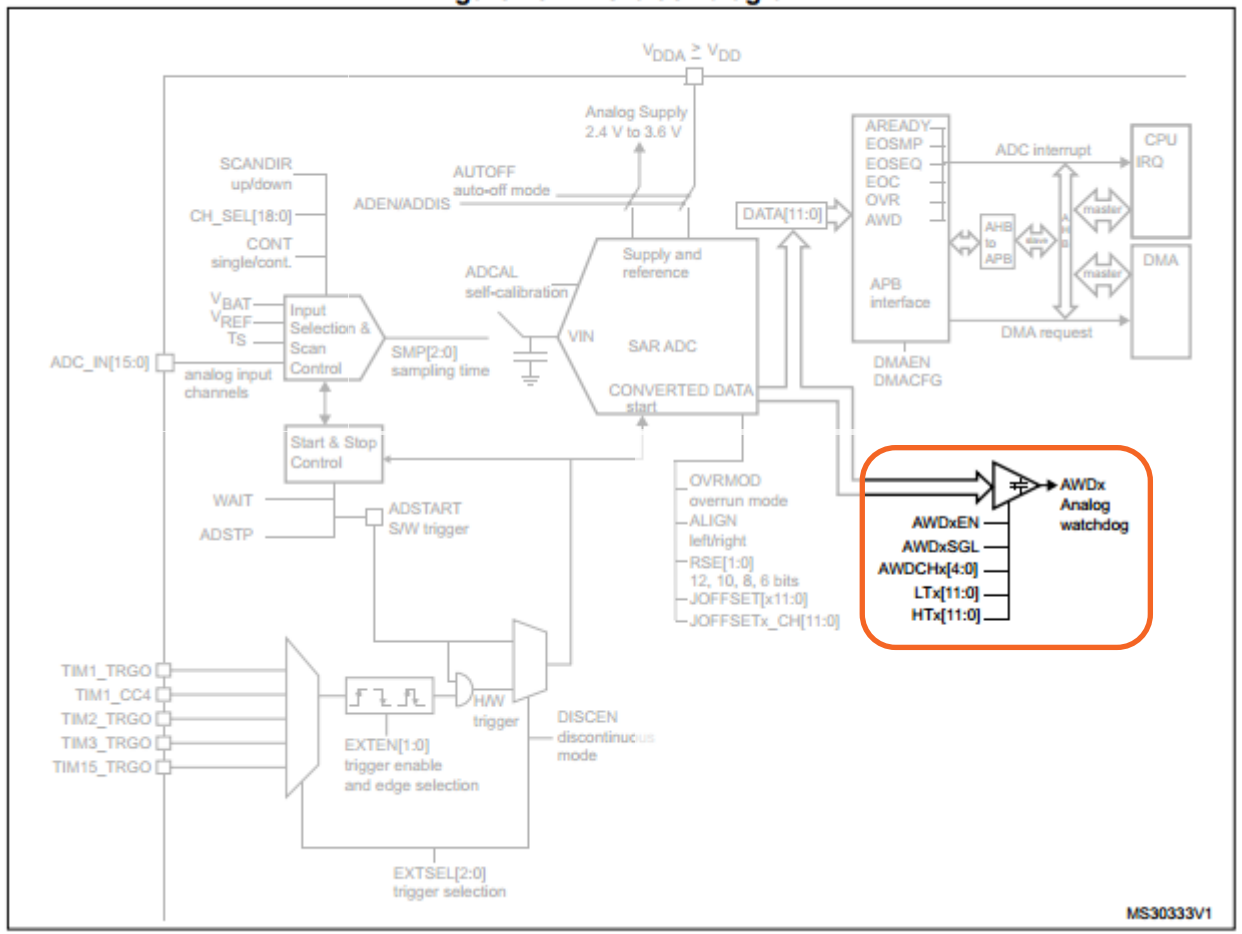


ADC

Analogue Watchdog Settings

- Used to detect event in the conversion parameters such as out of a user defined voltage range

Figure 25. ADC block diagram

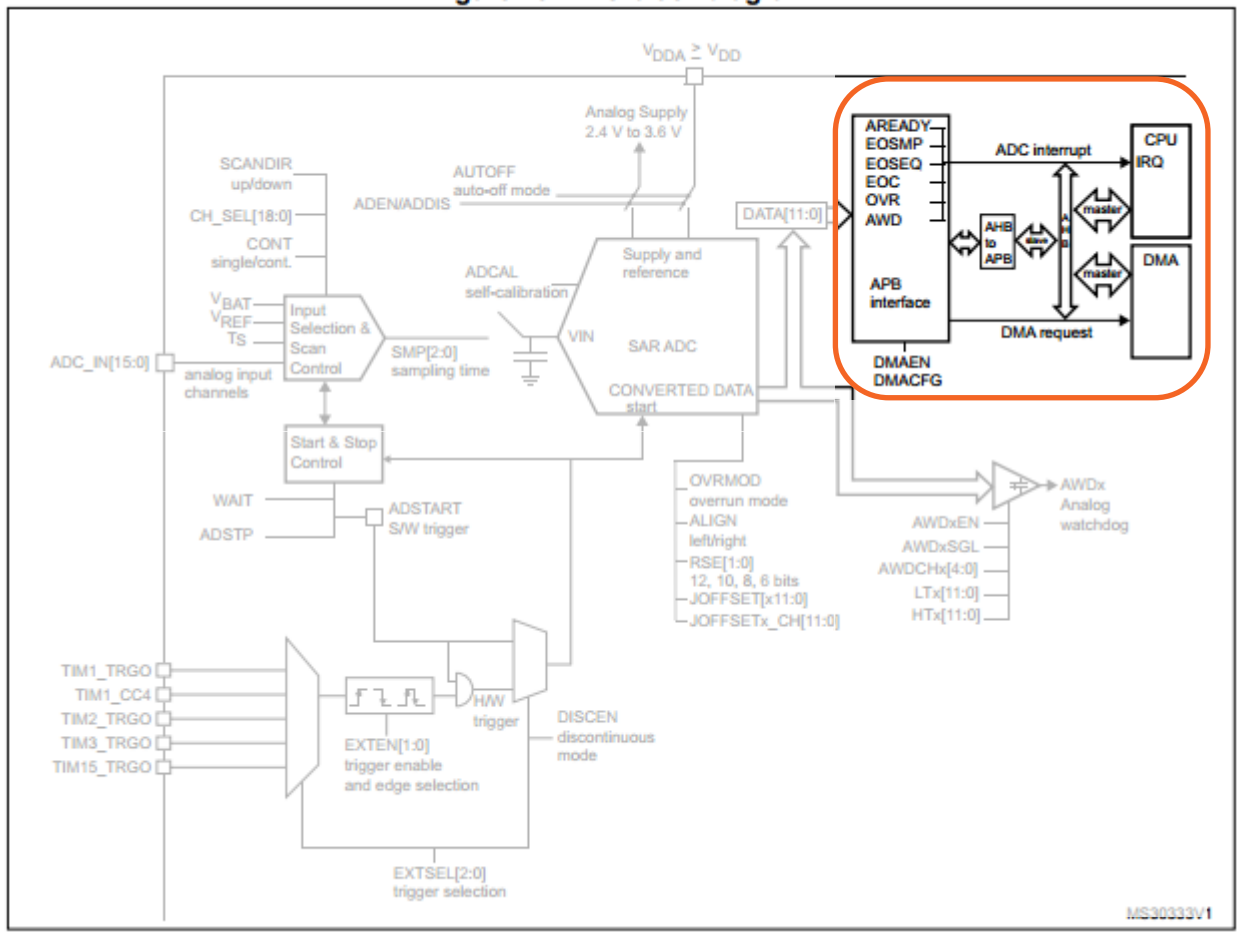


ADC

Data Output Settings

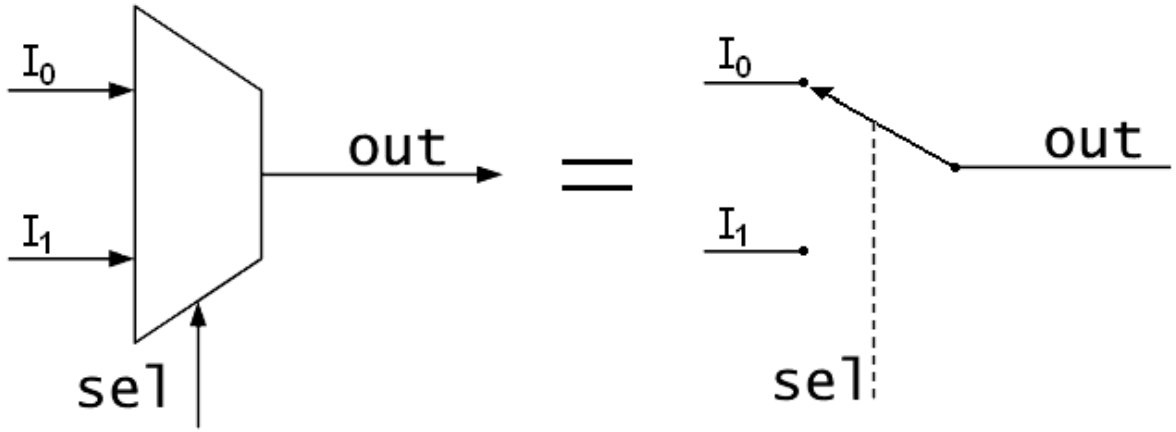
- Flags can be raised after various events.
- Interrupts can be generated.
- Data can be transferred using DMA.

Figure 25. ADC block diagram



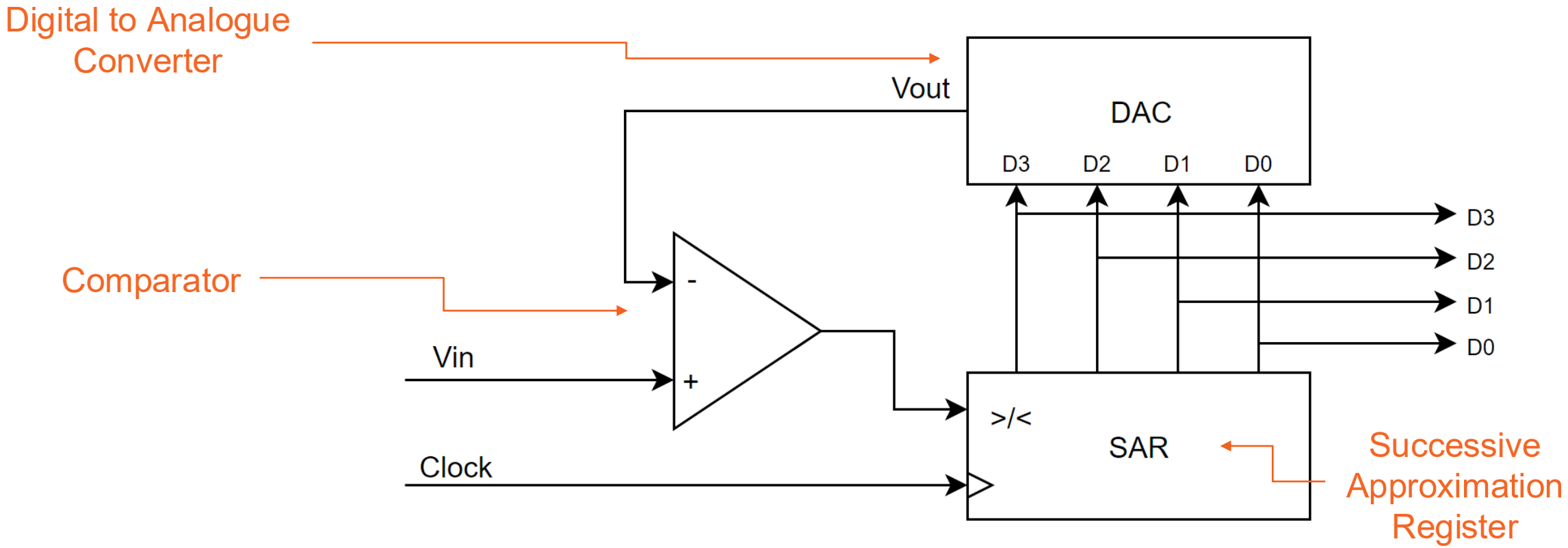
ADC

- Contains a 12 bit ADC
- 19 sample channels which can sample the analogue input pins (16 external, 3 internal) but only one converter
- The microcontroller multiplexers the input lines into the ADC



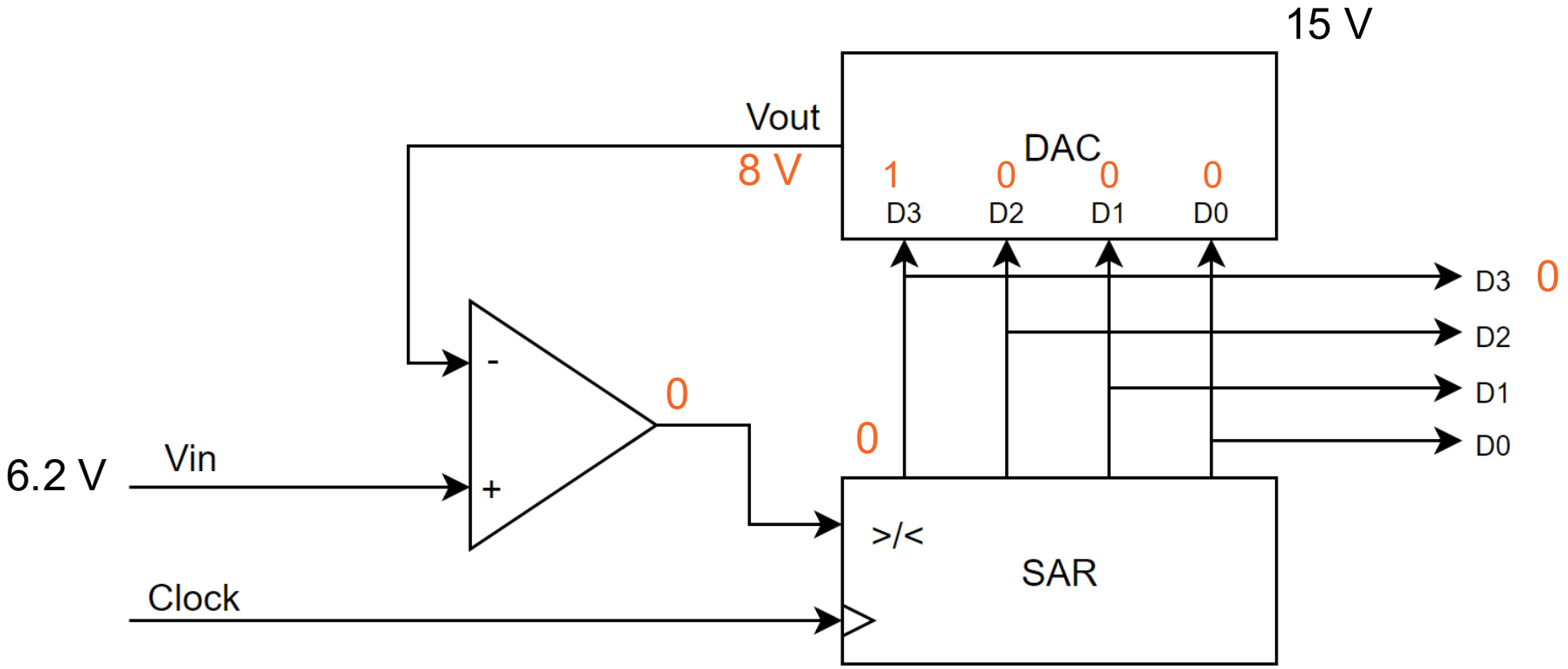
ADC

- The converter is a successive approximation type



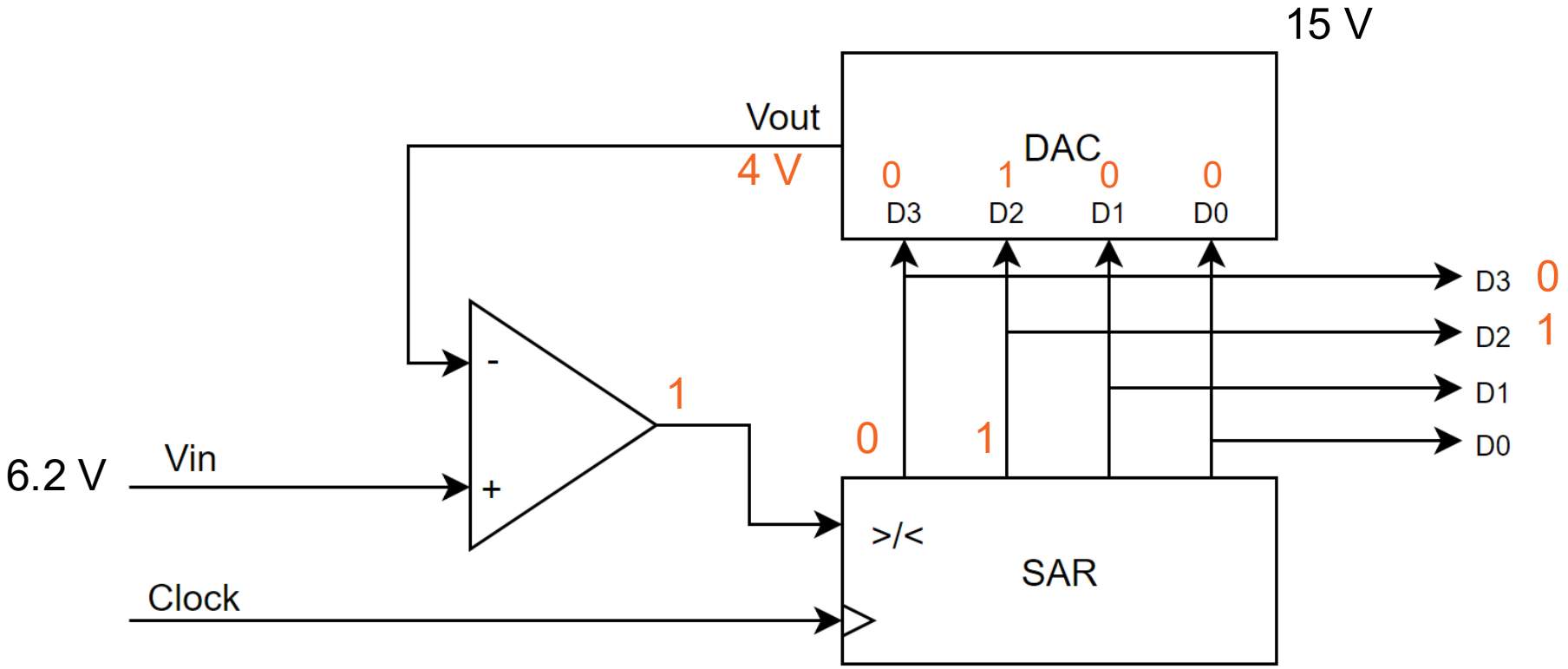
ADC

E.g.



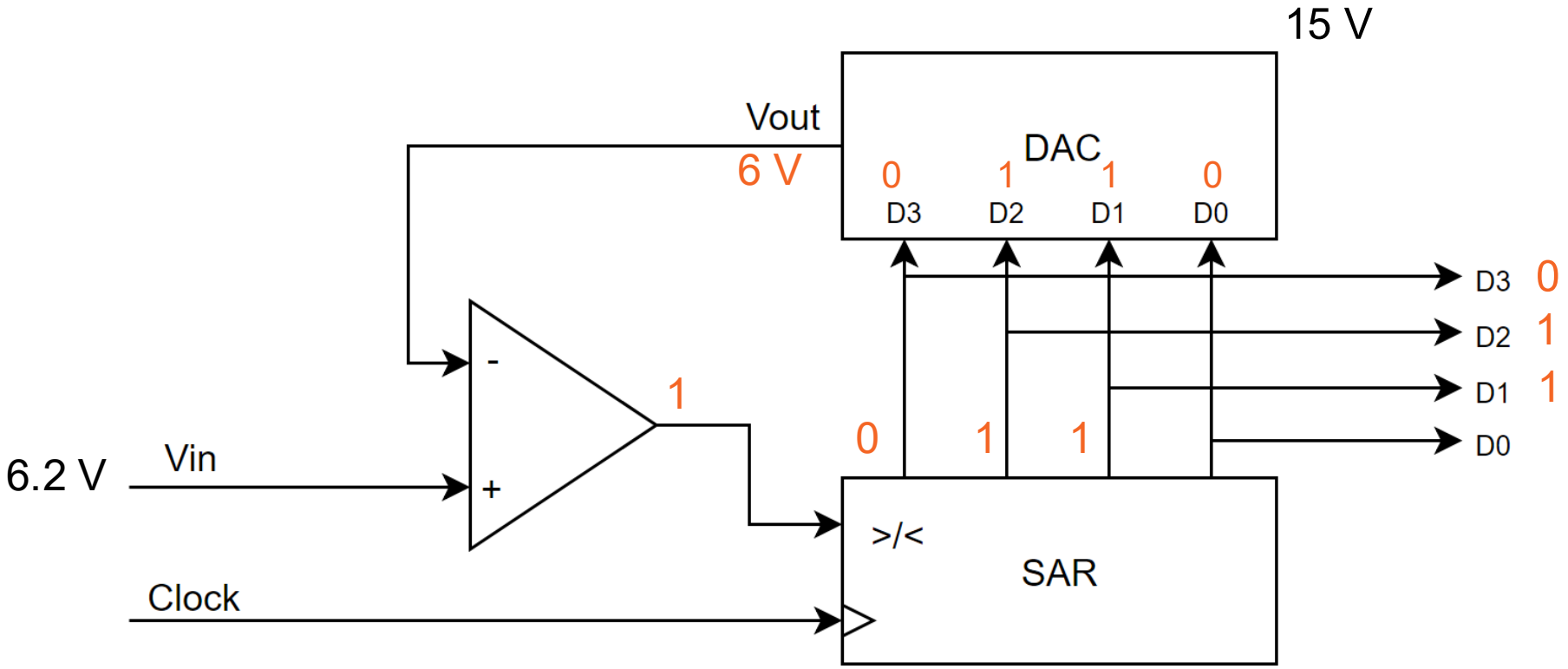
ADC

E.g.


15 V
Vout
4 V
DAC
0
1
0
0
D3
D2
D1
D0
D3
D2
D1
D0
0
1
-
+
1
0
1
6.2 V
Vin
>/<
SAR
Clock

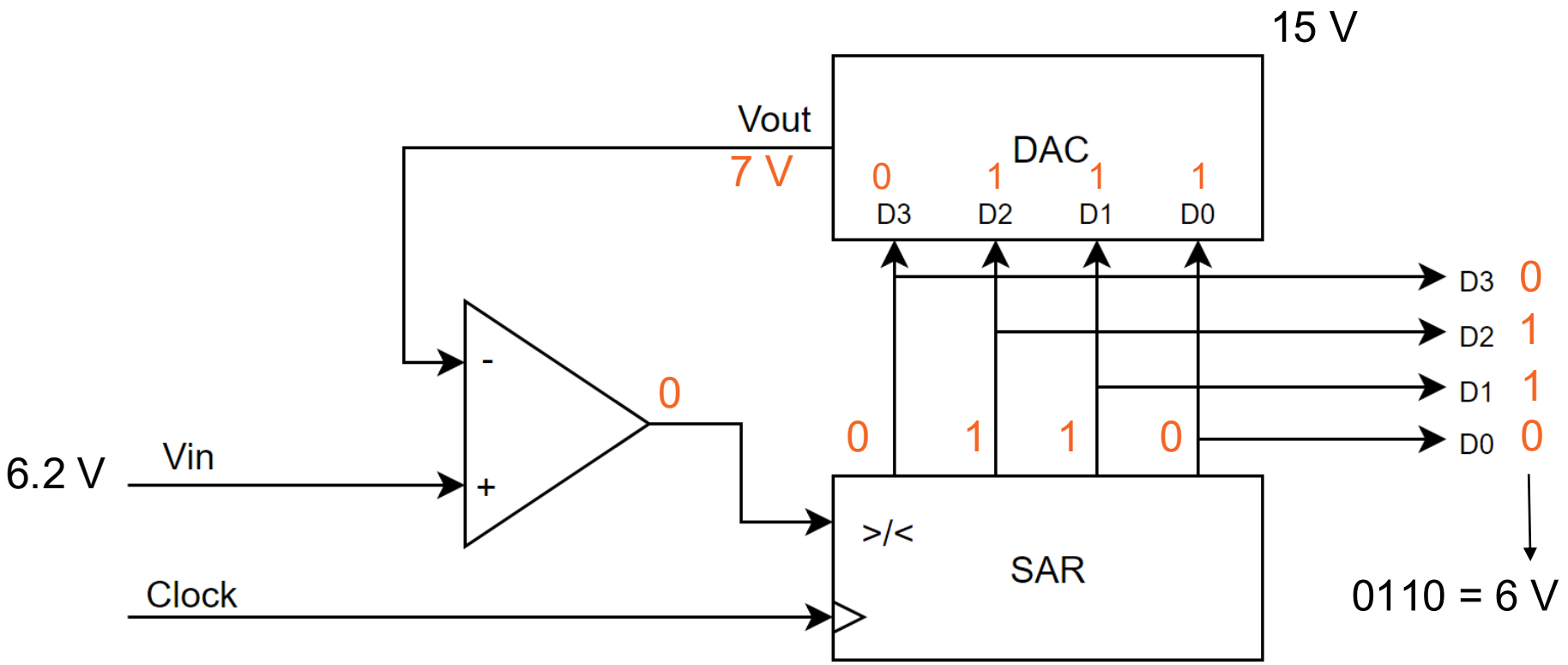
ADC

E.g.


15 V
Vout
6 V
DAC
0
1
1
0
D3
D2
D1
D0
D3 0
D2 1
D1 1
D0
-
+
1
0
1
1
6.2 V
Vin
>/<
SAR
Clock

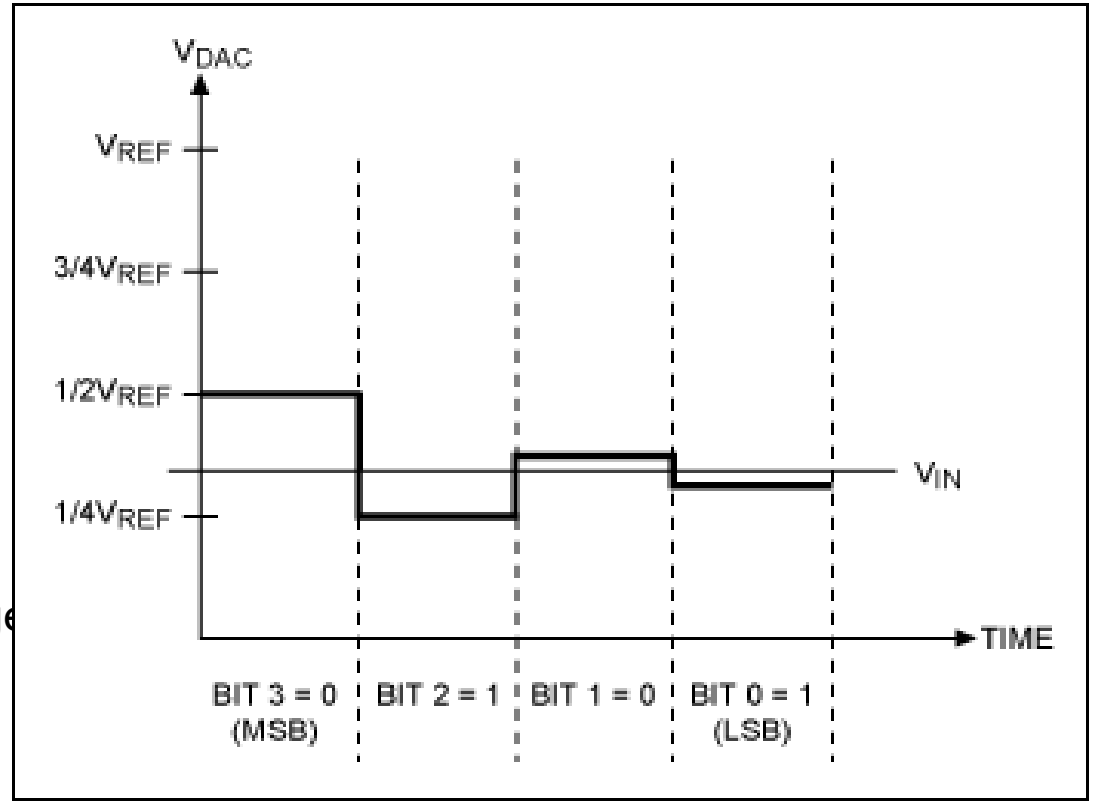
ADC

E.g.


15 V
Vout
7 V
DAC
0
1
1
1
D3
D2
D1
D0
D3 0
D2 1
D1 1
D0 0
-
+
0
6.2 V
Vin
0
1
1
0
>/<
SAR
Clock
0110 = 6 V

ADC

E.g.



- Can be trigge

ADC

Enabling

- Provide a clock to the ADC peripheral by setting the corresponding bit in the RCC_APB2ENR

```
RCC->APB2ENR |= RCC_APB2ENR_ADCEN;
```

- Activating the ADC by setting the ADEN bit in the ADC_CR

```
ADC1->CR |= ADC_CR_ADEN;
```

ADC

Selecting the Channel

- The ADC_CHSELR register controls which channel is selected. On our device there are 10 different pins which can be selected for use as an ADC channel. (Check Table 13 of the Datasheet in the ‘Additional functions’ column.) For example, PB1 is connected to ADC channel 9, whilst PA6 is connected to ADC channel 6.

- The pins must first be configured for use as analogue inputs. In order to set a pin to operate in analogue mode, both bits which control that pin in the GPIOx_MODER should be set to 1. (See Section 9.4.1 of the Reference Manual for more info on the modes.)

```
GPIOA->MODER |= GPIO_MODER_MODER6;
```

ADC

Selecting the Channel

- After the pins have been configured, select the channels to be sampled in the ADC_CHSELR register

```
ADC1->CHSELR |= ADC_CHSELR_CHSEL5
```

ADC

Configuration Settings

- Setup the Resolution - This is default 12 bits. You can set it to 8 using the following command

```
ADC1->CFGR1 |= ADC_CFGR1_RES_1;
```

- Setup the alignment - This is default right alignment and is set with the ALIGN bit in the ADC_CFGR1

```
ADC1->CFGR1 |= ADC_CFGR1_ALIGN;
```

ADC

Configuration Settings

- Wait until configuration is complete - When ADRDY = 1 the ADC is ready to begin converting

```
while((ADC1->ISR & ADC_ISR_ADRDY) == 0);
```

ADC

- 32 but wide busses, but the ADC output is 12 bits wide
- There are two available justifications, left and right which correspond to the ALIGN bit being set to 0 or 1 in ADC_CFRG1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	**b12**	**b11**	**b10**	**b9**	**b8**	**b7**	**b6**	**b5**	**b4**	**b3**	**b2**	**b1**	**b0**

Right Justified
ALIGN = 0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
b12	**b11**	**b10**	**b9**	**b8**	**b7**	**b6**	**b5**	**b4**	**b3**	**b2**	**b1**	**b0**	0	0	0

Left Justified
ALIGN = 1

ADC

- Often your accuracy does not need to be very high
- In these cases we can drop the resolution from 12 bits to 10, 8 or 6.
- ADC can be run faster if less bits are required
- ADC will convert at 100ms for 12 bit and 643ns for 6 bits

ADC

- Single shot conversion (CONT bit reset in ADC_CFGR1) and read sequence

1. Conversion sequence started by setting the ADSTART bit in ADC_CR or a hardware trigger.
2. Wait for the EOC bit in ADC_ISR to go high after each conversion. The next channel is automatically sampled at this point.
3. Read the result from ADC_DR. The result is in a format as defined by the resolution and alignment.
4. New conversion initiated by ADSTART or another trigger.

ADC

- Continuous conversion (CONT bit set in ADC_CFGR1) and read sequence

1. Conversion sequence started by setting the ADSTART bit in ADC_CR or a hardware trigger.
2. Wait for the EOC bit in ADC_ISR to go high after each conversion. The next channel is automatically sampled at this point.
3. Read the result from ADC_DR. The result is in a format as defined by the resolution and alignment.
4. New conversion sequence automatically triggered after all channels have been sampled.

ADC

- Wait Mode (WAIT bit set in ADC_CFGR1)

1. Single or continuous conversion sequence started by setting the ADSTART bit in ADC_CR or a hardware trigger.
2. Once EOC bit in ADC_ISR goes high, ADC pauses before next conversion is initiated.
3. Reading ADC_DR.
4. New conversion sequence automatically triggered after all channels have been sampled.

ADC CODE EXAMPLE

```
#define STM32F051

#include <stdint.h>
#include "stm32f0xx.h"

void main(void);
void init_leds(void);
void init_adc(void);

void main(void)
{
    init_leds();
    init_adc();

     while(1)
     {
                    // start conversion with ADSTART=1
                    ADC1->CR |= ADC_CR_ADSTART;
                    // wait for EOC == 1. Not necessary to clear EOC as we read from DR
                    while((ADC1->ISR & ADC_ISR_EOC) == 0);
                    GPIOB->ODR = ADC1->DR;
     }
}
```

ADC CODE EXAMPLE

```
void init_leds(void)
 {
        RCC->AHBENR |= RCC_AHBENR_GPIOBEN;          //enable clock for LEDs
        GPIOB->MODER |= GPIO_MODER_MODER0_0;        //set PB0 to output
        GPIOB->MODER |= GPIO_MODER_MODER1_0;        //set PB1 to output
        GPIOB->MODER |= GPIO_MODER_MODER2_0;        //set PB2 to output
        GPIOB->MODER |= GPIO_MODER_MODER3_0;        //set PB3 to output
        GPIOB->MODER |= GPIO_MODER_MODER4_0;        //set PB4 to output
        GPIOB->MODER |= GPIO_MODER_MODER5_0;        //set PB5 to output
        GPIOB->MODER |= GPIO_MODER_MODER6_0;        //set PB6 to output
        GPIOB->MODER |= GPIO_MODER_MODER7_0;        //set PB7 to output
}


void init_adc(void)
{
        RCC->APB2ENR |= RCC_APB2ENR_ADCEN;          //enable clock for ADC
        RCC->AHBENR |= RCC_AHBENR_GPIOAEN;          //enable clock for port
        GPIOA->MODER |= GPIO_MODER_MODER6;          //set PA6 to analogue mode
        ADC1->CR |= ADC_CR_ADEN;                    //set ADEN=1 in the ADC_CR register
        ADC1->CHSELR |= ADC_CHSELR_CHSEL6;//select channel 6
        ADC1->CFGR1 |= ADC_CFGR1_RES_1;             //resolution to 8 bit
        while((ADC1->ISR & ADC_ISR_ADRDY) == 0);    //wait until ADRDY==1 in ADC_ISR
}
```

ADC CODE EXAMPLE

```
#define STM32F051

#include <stdint.h>
#include "stm32f0xx.h"

void main(void);
void init_leds(void);
void init_adc(void);

void main(void)
{
    init_leds();
    init_adc();

     while(1)
     {
                    // start conversion with ADSTART=1
                    ADC1->CR |= ADC_CR_ADSTART;
                    // wait for EOC == 1. Not necessary to clear EOC as we read from DR
                    while((ADC1->ISR & ADC_ISR_EOC) == 0);
                    GPIOB->ODR = ADC1->DR;
     }
}
```